SQL Notes

Complete Guide with Examples

From basics to advanced: DDL, DML, Joins, Subqueries, Window Functions, Normalization, Transactions and more

Prepared by Waseem Abbas

#	Topic
1	Introduction to SQL
2	Sample Tables Used in Notes
3	Data Types
4	DDL Commands
5	DML Commands
6	SELECT, WHERE and Operators
7	Sorting, DISTINCT, LIMIT, Aliases
8	Aggregate Functions, GROUP BY, HAVING
9	Constraints and Keys
10	Joins
11	Set Operations
12	Subqueries
13	Built-in Functions and CASE
14	Views and Indexes
15	Normalization
16	Transactions, ACID and TCL
17	DCL Commands
18	CTE and Window Functions
19	Stored Procedures, Functions, Triggers
20	Order of Execution, Cheat Sheet, Interview Questions

1. Introduction to SQL

SQL (Structured Query Language) is the standard language used to create, read, update and delete data stored in a **Relational Database Management System (RDBMS)** such as MySQL, PostgreSQL, Oracle, SQL Server and SQLite.

Important Terms

Term	Meaning
Database	Organized collection of data.
Table (Relation)	Data stored in rows and columns.
Row (Record / Tuple)	One single entry in a table.
Column (Field / Attribute)	One property of the data, e.g. name, age.
Schema	Structure / blueprint of the database.
Primary Key	Column that uniquely identifies each row.
Foreign Key	Column that refers to the primary key of another table.

Categories of SQL Commands

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define / change structure
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Change data
DQL	Data Query Language	SELECT	Fetch data
DCL	Data Control Language	GRANT, REVOKE	Permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Note
SQL keywords are not case-sensitive (SELECT = select), but writing them in CAPITAL letters is good practice. Every statement normally ends with a semicolon (;).

2. Sample Tables Used in Notes

All examples in these notes use the following tables. Practice them in any SQL tool.

employees

emp_id	name	salary	dept_id	city	manager_id
1	Ali	50000	10	Lahore	NULL
2	Sara	65000	20	Karachi	1
3	Ahmed	45000	10	Islamabad	1
4	Fatima	80000	30	Lahore	2
5	Usman	50000	NULL	Multan	2

departments

dept_id	dept_name
10	IT
20	HR
30	Finance
40	Marketing

3. Data Types

Type	Description	Example
INT / BIGINT / SMALLINT	Whole numbers	25, 100000
DECIMAL(p,s) / NUMERIC	Exact decimal numbers	DECIMAL(8,2) = 123456.78
FLOAT / DOUBLE	Approximate decimals	3.14159
CHAR(n)	Fixed-length text	CHAR(3) = 'PAK'
VARCHAR(n)	Variable-length text	VARCHAR(50) = 'Ali Khan'
TEXT	Long text	Articles, comments
DATE	Date only (YYYY-MM-DD)	'2026-10-15'
TIME / DATETIME / TIMESTAMP	Time or date + time	'2026-10-15 14:30:00'
BOOLEAN	True / False	TRUE

4. DDL Commands

CREATE TABLE

```
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    salary     DECIMAL(10,2) DEFAULT 30000,
    dept_id    INT,
    city       VARCHAR(30),
    manager_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

ALTER TABLE

```
ALTER TABLE employees ADD email VARCHAR(100);          -- add column
ALTER TABLE employees DROP COLUMN email;               -- remove column
ALTER TABLE employees MODIFY name VARCHAR(80);         -- change type (MySQL)
ALTER TABLE employees ALTER COLUMN name TYPE VARCHAR(80); -- PostgreSQL
ALTER TABLE employees RENAME COLUMN city TO location;  -- rename column
ALTER TABLE employees RENAME TO staff;                 -- rename table
```

DROP, TRUNCATE, DELETE - Difference

```
DROP TABLE employees;       -- deletes table structure + data
TRUNCATE TABLE employees;   -- deletes ALL rows, structure remains
DELETE FROM employees;      -- deletes rows (can use WHERE)
```

Feature	DROP	TRUNCATE	DELETE
Type	DDL	DDL	DML
Removes	Table + data	All rows	Selected / all rows
WHERE clause	No	No	Yes
Rollback	No (usually)	Mostly no	Yes (in transaction)
Speed	Fast	Fast	Slower

5. DML Commands

INSERT

```
INSERT INTO departments VALUES (10, 'IT');
INSERT INTO employees (emp_id, name, salary, dept_id, city)
VALUES (1, 'Ali', 50000, 10, 'Lahore');

-- multiple rows
INSERT INTO departments (dept_id, dept_name)
VALUES (20,'HR'), (30,'Finance'), (40,'Marketing');

-- insert from another table
INSERT INTO emp_backup SELECT * FROM employees WHERE salary > 50000;
```

UPDATE

```
UPDATE employees SET salary = 55000 WHERE emp_id = 1;
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 10;  -- 10% raise
```

> **Warning**
> Always use WHERE with UPDATE and DELETE. Without it, ALL rows are changed or deleted!

DELETE

```
DELETE FROM employees WHERE emp_id = 5;
DELETE FROM employees WHERE city = 'Multan' AND salary < 40000;
```

6. SELECT, WHERE and Operators

SELECT

```
SELECT * FROM employees;                -- all columns
SELECT name, salary FROM employees;     -- specific columns
SELECT name, salary * 12 AS yearly_salary FROM employees;
```

WHERE Clause

```
SELECT * FROM employees WHERE city = 'Lahore';
SELECT * FROM employees WHERE salary > 50000;
```

Operators

Type	Operators
Comparison	= != (or <>) > < >= <=
Logical	AND, OR, NOT
Special	BETWEEN, IN, LIKE, IS NULL, EXISTS
Arithmetic	+ - * / %

```
-- AND / OR / NOT
SELECT * FROM employees WHERE city='Lahore' AND salary > 60000;
SELECT * FROM employees WHERE city='Lahore' OR city='Karachi';
SELECT * FROM employees WHERE NOT city='Lahore';

-- BETWEEN (inclusive)
SELECT * FROM employees WHERE salary BETWEEN 45000 AND 60000;

-- IN
SELECT * FROM employees WHERE city IN ('Lahore','Karachi');
SELECT * FROM employees WHERE city NOT IN ('Lahore','Karachi');

-- LIKE  (%  = any characters,  _ = exactly one character)
SELECT * FROM employees WHERE name LIKE 'A%';     -- starts with A
SELECT * FROM employees WHERE name LIKE '%a';     -- ends with a
SELECT * FROM employees WHERE name LIKE '%ma%';   -- contains ma
SELECT * FROM employees WHERE name LIKE '_li';    -- 3 letters ending with li

-- NULL check
SELECT * FROM employees WHERE dept_id IS NULL;
SELECT * FROM employees WHERE dept_id IS NOT NULL;
```

Important
NULL means unknown / no value. It is NOT zero or empty string. You cannot compare it with = ; always use IS NULL or IS NOT NULL. Any arithmetic with NULL gives NULL.

7. Sorting, DISTINCT, LIMIT, Aliases

```
-- ORDER BY (ASC is default)
SELECT * FROM employees ORDER BY salary DESC;
SELECT * FROM employees ORDER BY dept_id ASC, salary DESC;

-- DISTINCT removes duplicates
SELECT DISTINCT city FROM employees;

-- LIMIT / OFFSET (MySQL, PostgreSQL)
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;          -- top 3
SELECT * FROM employees ORDER BY salary DESC LIMIT 3 OFFSET 3; -- next 3

-- SQL Server:   SELECT TOP 3 * FROM employees ORDER BY salary DESC;
-- Oracle:       ... FETCH FIRST 3 ROWS ONLY;

-- Aliases
SELECT e.name AS employee_name, e.salary AS pay FROM employees AS e;
```

Nth Highest Salary (Very Common Question)

```
-- 2nd highest salary
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

```
-- Using subquery
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

8. Aggregate Functions, GROUP BY, HAVING

Function	Purpose
COUNT()	Number of rows. COUNT(*) counts all; COUNT(col) ignores NULLs
SUM()	Total of values
AVG()	Average
MIN() / MAX()	Smallest / largest value

```
SELECT COUNT(*)      FROM employees;          -- 5
SELECT SUM(salary)   FROM employees;          -- 290000
SELECT AVG(salary)   FROM employees;          -- 58000
SELECT MAX(salary), MIN(salary) FROM employees;
```

GROUP BY

Groups rows having the same value so that aggregate functions can be applied per group.

```
SELECT dept_id, COUNT(*) AS total_emp, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id;
```

dept_id	total_emp	avg_sal
10	2	47500
20	1	65000
30	1	80000
NULL	1	50000

HAVING

HAVING filters groups (after GROUP BY). **WHERE** filters rows (before GROUP BY).

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE city <> 'Multan'          -- filter rows first
GROUP BY dept_id
HAVING AVG(salary) > 50000;     -- filter groups after
```

WHERE	HAVING
Filters individual rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions	Can use aggregate functions

9. Constraints and Keys

Constraints

Constraint	Meaning
NOT NULL	Column cannot be empty.

Constraint	Meaning
UNIQUE	All values must be different (NULL allowed).
PRIMARY KEY	NOT NULL + UNIQUE. Only one per table.
FOREIGN KEY	Links to primary key of another table (referential integrity).
CHECK	Value must satisfy a condition.
DEFAULT	Value used when none is given.

```
CREATE TABLE students (
    id     INT PRIMARY KEY AUTO_INCREMENT,
    name   VARCHAR(50) NOT NULL,
    email  VARCHAR(100) UNIQUE,
    age    INT CHECK (age >= 16),
    status VARCHAR(10) DEFAULT 'active',
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE SET NULL ON UPDATE CASCADE
);
```

Types of Keys

Key	Description
Super Key	Any set of columns that uniquely identifies a row.
Candidate Key	Minimal super key (no extra column).
Primary Key	Candidate key chosen as main identifier.
Alternate Key	Candidate keys not chosen as primary.
Composite Key	Primary key made of 2 or more columns.
Foreign Key	Refers to primary key of another table.

10. Joins

A JOIN combines rows from two or more tables based on a related column.

INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

name	dept_name
Ali	IT
Ahmed	IT
Sara	HR
Fatima	Finance

Usman is missing (dept_id is NULL) and Marketing is missing (no employee).

LEFT JOIN

All rows from the left table + matching rows from right (NULL if no match).

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

name	dept_name
Ali	IT
Sara	HR
Ahmed	IT
Fatima	Finance
Usman	NULL

RIGHT JOIN

All rows from the right table + matching rows from left.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;   -- Marketing appears with NULL name
```

FULL OUTER JOIN

All rows from both tables; NULL where no match. (Not supported directly in MySQL; use UNION of LEFT and RIGHT JOIN.)

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

CROSS JOIN

Cartesian product: every row of table A with every row of table B (5 x 4 = 20 rows here).

```
SELECT e.name, d.dept_name FROM employees e CROSS JOIN departments d;
```

SELF JOIN

A table joined with itself. Example: employee and their manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Find rows with NO match

```
-- Employees who have no department
SELECT e.name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Join	Result
INNER	Only matching rows
LEFT	All left + matching right
RIGHT	All right + matching left
FULL	All rows from both
CROSS	All combinations
SELF	Table joined to itself

11. Set Operations

Combine results of two SELECT queries. Both must have the same number of columns with compatible data types.

```
SELECT city FROM employees
UNION            -- all unique rows (duplicates removed)
SELECT city FROM customers;

SELECT city FROM employees
UNION ALL        -- all rows including duplicates (faster)
SELECT city FROM customers;

SELECT city FROM employees
INTERSECT        -- only rows present in both
SELECT city FROM customers;

SELECT city FROM employees
EXCEPT           -- rows in first but not in second (MINUS in Oracle)
SELECT city FROM customers;
```

12. Subqueries

A subquery is a query inside another query, written in parentheses.

Single-value subquery

```
-- Employees earning more than average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Multi-value subquery (IN)

```
-- Employees working in IT or HR
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments
                  WHERE dept_name IN ('IT','HR'));
```

EXISTS

```
-- Departments that have at least one employee
SELECT dept_name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Correlated subquery

Inner query depends on the outer query and runs once for each outer row.

```
-- Employees earning more than the average of THEIR department
SELECT name, salary, dept_id FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees
                WHERE dept_id = e.dept_id);
```

Subquery in FROM (Derived table)

```
SELECT dept_id, avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees GROUP BY dept_id) AS t
WHERE avg_sal > 50000;
```

13. Built-in Functions and CASE

String Functions

```
SELECT UPPER('ali');                  -- ALI
SELECT LOWER('ALI');                  -- ali
SELECT LENGTH('Pakistan');            -- 8
SELECT CONCAT('Ali', ' ', 'Khan');    -- Ali Khan
SELECT SUBSTRING('Pakistan', 1, 3);   -- Pak
SELECT TRIM('  hello  ');             -- hello
SELECT REPLACE('SQL is hard','hard','easy');  -- SQL is easy
SELECT LEFT('Pakistan', 3), RIGHT('Pakistan', 3);   -- Pak, tan
```

Numeric Functions

```
SELECT ROUND(123.456, 2);   -- 123.46
SELECT CEIL(4.2);           -- 5
SELECT FLOOR(4.8);          -- 4
SELECT ABS(-10);            -- 10
SELECT MOD(10, 3);          -- 1
SELECT POWER(2, 3);         -- 8
```

Date Functions (MySQL)

```
SELECT NOW();                          -- current date and time
SELECT CURDATE();                      -- current date
SELECT YEAR('2026-10-15');             -- 2026
SELECT MONTH('2026-10-15');            -- 10
SELECT DATEDIFF('2026-12-31','2026-10-15');   -- 77
SELECT DATE_ADD('2026-10-15', INTERVAL 7 DAY);
```

NULL Handling

```
SELECT COALESCE(dept_id, 0) FROM employees;     -- first non-NULL value
SELECT IFNULL(dept_id, 0) FROM employees;       -- MySQL
SELECT NULLIF(10, 10);                          -- NULL (if both equal)
```

CASE Expression (if-else in SQL)

```
SELECT name, salary,
  CASE
    WHEN salary >= 70000 THEN 'High'
    WHEN salary >= 50000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_level
FROM employees;
```

name	salary	salary_level
Ali	50000	Medium
Sara	65000	Medium
Ahmed	45000	Low
Fatima	80000	High

14. Views and Indexes

View

A view is a **virtual table** based on a SELECT query. It stores the query, not the data. Used for simplicity and security.

```
CREATE VIEW it_employees AS
SELECT emp_id, name, salary FROM employees WHERE dept_id = 10;

SELECT * FROM it_employees;          -- use like a table
DROP VIEW it_employees;
```

Index

An index is a data structure (usually B-Tree) that makes searching faster, like the index of a book. It speeds up SELECT but slows down INSERT / UPDATE / DELETE a little and uses extra space.

```
CREATE INDEX idx_name ON employees(name);
CREATE UNIQUE INDEX idx_email ON students(email);
CREATE INDEX idx_dept_sal ON employees(dept_id, salary);   -- composite index
DROP INDEX idx_name ON employees;
```

Tip
Create indexes on columns used often in WHERE, JOIN, ORDER BY. Primary key and unique columns get an index automatically.

15. Normalization

Normalization is organizing data to **reduce redundancy** and avoid anomalies (insert, update, delete problems).

Normal Form	Rule	Example Problem Fixed
1NF	Every cell has a single (atomic) value; no repeating groups.	Phone column containing '0300..., 0321...'
2NF	1NF + no partial dependency (non-key column must depend on the WHOLE primary key).	student_name depends only on student_id in table (student_id, course_id)
3NF	2NF + no transitive dependency (non-key column must not depend on another non-key column).	dept_name depends on dept_id which depends on emp_id
BCNF	For every dependency X -> Y, X must be a super key.	Stronger version of 3NF

Example

Unnormalized: orders(order_id, customer_name, customer_city, product, price)
Customer data repeats on every order. **Fix:** split into three tables:

```
customers(customer_id PK, customer_name, customer_city)
products(product_id PK, product, price)
orders(order_id PK, customer_id FK, product_id FK)
```

Denormalization
Intentionally adding redundancy to improve read speed (used in reporting / data warehouses).

16. Transactions, ACID and TCL

A **transaction** is a group of SQL statements treated as one unit: either all succeed or none.

ACID Properties

Property	Meaning
Atomicity	All or nothing.
Consistency	Database moves from one valid state to another.
Isolation	Concurrent transactions do not interfere with each other.
Durability	Once committed, data is permanent even after a crash.

Example: Bank Transfer

```
START TRANSACTION;   -- or BEGIN;

UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 2;

COMMIT;              -- save permanently
-- ROLLBACK;         -- undo everything if something went wrong

SAVEPOINT sp1;             -- set a checkpoint
ROLLBACK TO sp1;           -- undo up to the checkpoint
```

Isolation Problems

Problem	Description
Dirty Read	Reading data that another transaction has not committed yet.
Non-repeatable Read	Same row read twice gives different values.
Phantom Read	Same query returns different set of rows.

Isolation levels (low to high): READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE.

17. DCL Commands

```
GRANT SELECT, INSERT ON employees TO user1;    -- give permission
GRANT ALL PRIVILEGES ON company.* TO admin1;
REVOKE INSERT ON employees FROM user1;         -- take back permission
```

18. CTE and Window Functions

CTE (Common Table Expression)

A temporary named result set defined using WITH. Makes complex queries readable.

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

Window Functions

Perform calculations across related rows **without collapsing** them into one row (unlike GROUP BY).
Syntax: **function() OVER (PARTITION BY ... ORDER BY ...)**

```
SELECT name, salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
  RANK()       OVER (ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

name	salary	row_num	rnk	dense_rnk
Fatima	80000	1	1	1
Sara	65000	2	2	2
Ali	50000	3	3	3
Usman	50000	4	3	3
Ahmed	45000	5	5	4

ROW_NUMBER always gives unique numbers. **RANK** gives same rank for ties and skips the next numbers. **DENSE_RANK** gives same rank for ties and does NOT skip.

```
-- Rank within each department
SELECT name, dept_id, salary,
  RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;

-- Previous / next row value, running total
SELECT name, salary,
  LAG(salary)  OVER (ORDER BY emp_id) AS prev_salary,
  LEAD(salary) OVER (ORDER BY emp_id) AS next_salary,
  SUM(salary)  OVER (ORDER BY emp_id) AS running_total
FROM employees;

-- Top 2 earners in each department
SELECT * FROM (
  SELECT name, dept_id, salary,
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk
  FROM employees) t
WHERE rk <= 2;
```

19. Stored Procedures, Functions, Triggers

Stored Procedure

A saved block of SQL code that can be executed again and again.

```
DELIMITER //
CREATE PROCEDURE get_emp_by_city(IN p_city VARCHAR(30))
BEGIN
    SELECT * FROM employees WHERE city = p_city;
END //
DELIMITER ;

CALL get_emp_by_city('Lahore');
```

User-Defined Function

Similar to a procedure but it must return a value and can be used inside SELECT.

```
DELIMITER //
CREATE FUNCTION yearly_salary(p_sal DECIMAL(10,2)) RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN p_sal * 12;
```

```
END //
DELIMITER ;

SELECT name, yearly_salary(salary) FROM employees;
```

Trigger

Code that runs automatically when an INSERT, UPDATE or DELETE happens on a table.

```
CREATE TRIGGER trg_salary_log
AFTER UPDATE ON employees
FOR EACH ROW
INSERT INTO salary_log (emp_id, old_salary, new_salary, changed_on)
VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
```

Feature	Procedure	Function
Return value	Optional	Must return
Use in SELECT	No	Yes
Called by	CALL / EXEC	Inside queries

20. Order of Execution, Cheat Sheet, Interview Questions

Logical Order of Execution

SQL is written in one order but executed in another:

Order	Clause
1	FROM / JOIN
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT
6	DISTINCT
7	ORDER BY
8	LIMIT / OFFSET

This is why you cannot use a SELECT alias inside WHERE (but you can in ORDER BY).

Quick Syntax Cheat Sheet

```
SELECT   columns / aggregates
FROM     table
JOIN     other_table ON condition
WHERE    row_condition
GROUP BY columns
HAVING   group_condition
ORDER BY columns [ASC|DESC]
LIMIT    n;
```

Common Interview Questions

Question	Short Answer
Difference between WHERE and HAVING?	WHERE filters rows before grouping; HAVING filters groups after grouping.
DELETE vs TRUNCATE vs DROP?	DELETE removes selected rows (DML); TRUNCATE removes all rows; DROP removes the whole table.
Primary key vs Unique key?	Primary: no NULL, only one per table. Unique: NULL allowed, many per table.
UNION vs UNION ALL?	UNION removes duplicates; UNION ALL keeps them and is faster.
CHAR vs VARCHAR?	CHAR is fixed length; VARCHAR is variable length.
RANK vs DENSE_RANK?	RANK skips numbers after ties; DENSE_RANK does not.
What is a view?	A virtual table based on a stored SELECT query.
What is an index?	Structure that speeds up searching of data.
COUNT(*) vs COUNT(column)?	COUNT(*) counts all rows; COUNT(column) ignores NULL values.
What is a foreign key?	A column referencing the primary key of another table to maintain integrity.
What is normalization?	Organizing tables to reduce redundancy and anomalies (1NF, 2NF, 3NF, BCNF).

Practice Questions

- Find the employees whose salary is above the company average.
- Display the department name along with the number of employees in it (include departments with 0 employees).
- Find the 3rd highest salary using DENSE_RANK.
- Find duplicate names: `SELECT name, COUNT(*) FROM employees GROUP BY name HAVING COUNT(*) > 1;`
- Show each employee with their manager name (self join).
- Delete duplicate rows keeping the one with the smallest id.

Good luck!
Practice daily on a real database (MySQL / PostgreSQL / SQLite) or on sites like W3Schools, LeetCode and HackerRank. Writing queries yourself is the best way to learn SQL.